Complexitylib:
Human-readable proofs from the Lean formalization

Working outline and proof translations
19 October 2026

This is an annotated plan with Cook–Levin and sparse-counter proof translations. The paper is intended as an AI-reviewed explanation of the arguments in Lean; the other chapters are proposed. The author controls selection and wording. Cook–Levin links refer to revision `263e7321`; sparse-counter links identify the implementation accompanying that section.

Annotated plan for a proof companion

Purpose. Translate the arguments in the Lean development into mathematical prose that can be read without following tactic scripts. The exposition should explain how each conclusion follows from the definitions and intermediate results actually used in its formal proof. The translations below cover the Cook–Levin proof route and sparse-counter signed powers. The other entries are proposed scope, not claims of completed translations.

A common form for each proof

State the exact theorem, its model and hypotheses. Introduce the objects needed to read its argument. Then explain the intermediate claims, their proofs and how they compose, with links to the corresponding Lean declarations. Make uses of choice, finiteness, encoding conventions and quantitative bounds explicit where they matter. Cite the mathematical sources and credit earlier formalizations. A named imported lemma can be used as a lemma, with its contract stated and a link for following the argument further.

For discussion. How far down should each translation go? The sample explains the tableau induction and its one-hot argument, while treating the machine's local step-equation lemma and previously certified list operations as named prerequisites.

1. Computation and reductions

Explain the machine and encoding contracts needed by later proofs; translate the composition and iteration arguments behind resource certificates. Use Cook–Levin as the first complete route from a mathematical construction to a class-level result. Subsequent sections could translate the universal simulation, selected complete-problem reductions and the time hierarchy, stating the precise overhead in each case.

For discussion. We can begin with Cook–Levin and introduce foundational lemmas on demand, or place a short model chapter first.

2. Space, alternation and interaction

Translate the recursive reachability argument for the polynomial-space Savitch result, the two containments giving $\mathsf{AP} = \mathsf{PSPACE}$, and the game evaluation behind $\mathsf{IP} \subseteq \mathsf{PSPACE}$. Explain the actual stored state and recursion bounds. Follow any characterization theorem used by these proofs back to its machine interface.

For discussion. Keep these as separate proofs with cross-references to shared space lemmas, or group them around the common problem of evaluating large computation trees.

3. Randomness, counting and cryptography

Translate the covering argument in Sipser–Gács–Lautemann; the isolation, parity and counting steps in Toda; and the hybrid and reduction arguments used for PRGs and adaptive PRFs. State the oracle, adversary and probability spaces used by the formal proofs. Follow each claimed efficiency guarantee to its certificate.

For discussion. Which proof should follow Cook–Levin? Toda would exercise quantitative probabilistic reasoning; a PRG/PRF proof would exercise the security interfaces.

4. Circuits, logic and combinatorial bounds

The sparse-counter signed-power argument is translated below. Further chapters could include Fagin's theorem, determinant circuits in VP, resolution size–width, and expander-walk hitting. For each, explain the construction or inequality chain in the actual Lean development. Keep formal polynomials distinct from polynomial functions, and proof certificates distinct from runtime analyses.

For discussion. Choose a small initial set for depth. The remaining results can stay in this coverage plan until their arguments have been translated and reviewed.

Priority gaps. First-order reductions and their restricted projection forms are a priority, building on the existing interpretations and formula pullbacks. Circuit Value is currently proved in P; its logspace P-completeness theorem remains to be formalized. The NC/AC/TC hierarchies are nonuniform; uniform-family and machine-characterization theorems are separate work. Likewise, the verifier definitions of #P and GapP still need their nondeterministic-machine characterizations. These are development targets, not translated proofs already present in the library.

5. The PCP theorem

Translate the route through gap amplification and composition to $\mathsf{NP} = \mathsf{PCP}(O(\log n), O(1))$, including the parameter calculations and the certificates for construction and verification. Preserve the proof's dependencies through explicit lemma references. Explain where the large fixed constants arise.

For discussion. This is likely a sequence of proof sections with its own dependency map, rather than one overview chapter.

Review and maintenance

An independent AI reviewer compares each translation with the cited Lean definitions, statements and proof bodies at a recorded revision. The review checks hypotheses, quantifier order, intermediate claims, proof steps and resource bounds, and records any omission as an explicit prerequisite or an unresolved gap. AI review is evidence of inspection; Lean does not check equivalence between the prose and the formal proof. The author continues to control wording, organization and selection.

Every AI-authored PR must synchronize the affected exposition with its final Lean changes and obtain a fresh independent review of the changed translation. A proof refactor may change the argument even when the theorem statement stays fixed. New mathematical developments need corresponding exposition; renamed or moved results need repaired links. A PR with no mathematical or exposition impact must say so explicitly. Its review must not claim coverage that has not been written.

Coverage and navigation. A small index will associate translated arguments with their source modules and revision, named prerequisites, and review status. It should make the boundary between reviewed exposition and proposed chapters visible. This preserves the old blueprint's links from readable mathematics to Lean, while giving the proofs themselves room to be read.

1 Cook–Levin: the argument in the formalization

This sample translates the route through the tableau, correctness, reduction and polynomial-time files to the final completeness theorem at revision `263e7321`. Its mathematical sources are the tableau presentations of Arora and Barak [1, Section 2.3] and Papadimitriou [3, Theorem 8.2]. The underlying local transition lemma and compositional efficiency rules are used as named prerequisites below; their internal proofs are not translated here.

Theorem (Formalized Cook–Levin theorem)**.** *SAT, with CNF encoding* encCNF*, is NP-complete under the library's tidy polynomial-time many-one reductions. UNSAT is coNP-complete under those reductions.*

1.1 The tableau and the statement it must satisfy

Fix a deterministic binary multi-tape machine M with finite control, a word u, a certificate length L, and a time bound T. The machine has a read-only input tape, finitely many two-way work tapes and a write-only output tape. Write $C_t(w)$ for its configuration after t steps on input $u +\!\!+ w$. The tableau formula $F(M, u, L, T)$ describes times $0, \ldots, T$. Its variables are $X_{t,\phi}$ for machine features ϕ, and W_j for certificate bits. The basic features record the control state, input-head position, work-head positions, work-cell symbols and a view of the output. Work positions are restricted to $[-T, T]$: a head starting at zero moves at most one cell per step. Auxiliary features record the symbols read and the current *situation*, meaning the active control state together with the symbols under the heads.

The formula is the conjunction of the following clause families:

1. Initial clauses set the start state, input position 1, work positions 0, blank work cells, and empty output.
2. At every time, one-hot clauses require exactly one feature in each basic group: one state, one head position, one symbol per represented cell, and one output view.
3. Before time T, read clauses force the actual symbols under the heads to be recorded. For the input tape they consult u, the variables W_j, or an end marker. Situation clauses force the active state and read symbols to give their situation.
4. Step clauses encode implications from terms describing the current configuration to basic features of the next one.
5. Final clauses require halted state and the output view corresponding to [true].

These are the constructors in `tableau`. The intended conclusion is

$$F(M, u, L, T) \text{ is satisfiable} \quad \Longleftrightarrow \quad \exists w \in \{0,1\}^L,\ C_T(w) \text{ is halted with output [true]}. \tag{1}$$

1.2 Why the one-hot constraints suffice

For an assignment σ, recover w by taking $\sigma(W_j)$ for $j < L$. Define $\mathrm{Agrees}(\sigma, t, C)$ to mean that for every basic feature ϕ,

$$\sigma(X_{t,\phi}) = \text{true} \quad \Longleftrightarrow \quad \phi \text{ holds of } C.$$

Each basic group has exactly one feature that holds of an actual configuration, provided its heads lie in the represented window. Suppose the assignment satisfies the one-hot clauses and assigns true to every basic feature that holds of C. In a group, let ψ be the unique feature holding of C. Its variable is true; exactly-one forces every other variable in the group false. Hence the assignment

agrees with C on the entire group. Applying this to every group proves `Agrees.of_oneHot`. This is why the later induction only needs to force the features that really occur.

1.3 From a satisfying assignment to an accepting run

Assume σ satisfies the tableau. We prove by induction that it agrees with $C_t(w)$ for every $t \le T$. The work-head bound holds at all these times because $|\mathrm{head}_i(C_t(w))| \le t \le T$. At time zero, the initial clauses set every basic feature that holds of the initial configuration. The one-hot argument therefore gives agreement at time zero.

For the induction step, suppose there is agreement at time $t < T$. The read clauses force the symbols actually read in $C_t(w)$: on a work tape, use its true head-position and cell-symbol features; on the input tape, use its true input position and the reconstructed certificate. The situation clauses then force the actual active situation when one holds.

Now let ϕ be a basic feature holding of $C_{t+1}(w)$. The local machine lemma `Feature.holds_step_iff_exists_stepTerms` says that some conjunction in $\mathrm{stepTerms}(M, \phi)$ holds of $C_t(w)$. For each literal in that conjunction, either its feature is basic, in which case agreement supplies the required truth value, or it is a positive situation literal, which the preceding paragraph supplies. Thus its step-clause antecedent is true, and the clause forces $\sigma(X_{t+1,\phi})$ true. The one-hot argument upgrades these positive facts to full agreement at time $t+1$. This is `Agrees.step`; induction gives `agrees_of_eval_tableau`.

The direction of the auxiliary constraints matters. They force the read symbols and situations that actually occur, but need not rule out every extra auxiliary truth value. The proof works because nonbasic literals in step terms are positive situation literals. Agreement itself concerns basic features. At time T, agreement turns the two final clauses into the actual halted state and output [true]. This proves the forward direction of (1).

1.4 From an accepting run to a satisfying assignment

Conversely, suppose w has length L and $C_T(w)$ is halted with output [true]. Set each certificate variable to its bit in w, and set $X_{t,\phi}$ true exactly when ϕ holds of $C_t(w)$. This is `runAssignment`. Its reconstructed certificate is w, by equality of the individual bits.

We check each clause family. The initial configuration supplies the initial clauses. Existence and uniqueness of the feature holding in each basic group give the one-hot clauses. A head reads the symbol at its position, which supplies the read clauses; the definition of a situation supplies the situation clauses. For a step clause, if its antecedent conjunction is true, the reverse direction of the local step-equation lemma says its consequent feature holds at the next time. Finally, the assumed halted state and output supply the final clauses. These checks are `eval_tableau_runAssignment`.

The two directions first prove an equivalence with the certificate fixed (`exists_eval_tableau_iff`). Existentially quantifying that certificate gives `tableau_satisfiable_iff`, which is exactly (1).

1.5 Instantiating the tableau with an NP verifier

Let A be any problem in NP under an encoding enc. Unpack its witness characterization: there are a polynomial p, a Boolean verifier V, and a tidy polynomial-time machine computing V, such that $a \in A$ exactly when some witness of length $p(|\mathrm{enc}(a)|)$ is accepted. The completeness proof extracts fixed naturals c, d that bound the verifier's time by $c(m+1)^d$ on encoded pairs of length m.

Put $n = |\mathrm{enc}(a)|$ and $L = p(n)$. The pair encoding frames the instance and appends the witness, so its length is $m = 2n + 1 + L$. Choose

$$u = \mathrm{frame}(\mathrm{enc}(a)), \qquad T = c(2n + 1 + p(n) + 1)^d.$$

Construct $F(M, u, L, T)$ and relabel its variables injectively by natural numbers. Injectivity preserves satisfiability: an assignment pulls back along the numbering, and any assignment to the original variables extends to their image and then to all numbered variables. The library applies `CNF.satisfiable_relabel` here. By (1), the resulting formula is satisfiable exactly when some length-L witness makes the machine halt with output true. The verifier certificate identifies its output with $[V(a, w)]$, so the witness condition is exactly $a \in A$. This proves `reduce_mem_SAT_iff_of_computesTidily`.

1.6 Certifying the reduction's resource bound

The efficiency proof follows the same construction. First compute (u, L, T) from a, representing the two numeric bounds in unary. Framing, encoded length, polynomial evaluation and the fixed power expression have polynomial-time certificates, composed in `computableTidilyInPolyTime_parameters`.

Next construct the tableau. Its clause constructors use bounded lists of times, positions and features. Each clause-family constructor has a certificate in `CookLevin/Clauses.lean`. Unfolding `tableau` expresses it as the concatenation of these families and bounded list traversals. The `polytime` tactic combines their registered certificates with the list-operation rules; this proves `computableTidilyInPolyTime_tableau`. The final certificate composes parameter computation, tableau construction and binary variable relabelling, in that order. Thus it accounts for constructing and encoding the reduction's output.

The machine, tape count, finite state space and polynomial bounds are fixed in this argument. Its finite transition and enumeration choices may therefore be stored in fixed tables. This explains why Lean definitions marked `noncomputable` still have certificates asserting the existence of machines with polynomial bounds for all encoded instances. The proof does not extract a particular runnable implementation of those choices.

1.7 The final completeness step

For arbitrary $A \in \mathsf{NP}$, the preceding correctness equivalence and efficiency certificate supply a polynomial-time many-one reduction to SAT. This is `SAT_npHard`. The separately certified SAT verifier gives `SAT.mem_NP`. Pairing these two facts proves `SAT_npComplete`. The generic complementation theorem then turns SAT completeness into `UNSAT_coNPComplete`.

2 Signed powers of sparsely modified counters

This section translates the sparse-displacement argument ported from Schlesinger [4], source revision `c4cfa198`. The source credits Moreno and Rivera [2] for the related block characterization of almost-commuting permutations; here the proof uses elementary defect containment and accounts for its Boolean-circuit cost. Write $W_n = \mathbb{Z}/2^n\mathbb{Z}$, represented by Lean's n-bit words, and $c(x) = x+1$. All word addition and subtraction below are modulo 2^n. For maps f, g on a finite set, put $D(f,g) = \{x : f(x) \neq g(x)\}$.

Theorem. *Suppose $h : W_n \to W_n$ is injective and commutes with $f : W_n \to W_n$. If $r = |D(f,c)|$, there exists a De Morgan circuit computing h, on the least-significant-bit-first representation of words, with at most $100(n+1)(r+1)$ gates. In particular, when f is a permutation the same bound holds for f^T for every fixed integer T, including negative T.*

This is a nonuniform existence theorem: the circuit may depend on the entire map and on T. No algorithm for efficiently discovering its constants or correction locations is proved. The theorem permits $n = 0$, when W_n has one element. The formal endpoints are `sparse_counter_centralizer` and `sparse_counter_power` in the circuit module.

2.1 Localizing the failures of commutation

The first step works for arbitrary maps on any type. If $hf = fh$, and both $f(x) = g(x)$ and $f(h(x)) = g(h(x))$, then

$$h(g(x)) = h(f(x)) = f(h(x)) = g(h(x)).$$

Consequently the set of commutation failures for h and g is contained in $D(f,g) \cup h^{-1}(D(f,g))$. On a finite type, injectivity of h gives $|h^{-1}(D(f,g))| \leq |D(f,g)|$: the image of this preimage injects into $D(f,g)$. The union bound now gives at most $2|D(f,g)|$ failures. These are `commutation_defect_subset` and `commutation_defect_card` in the finite-map module.

Apply this with $g = c$. Define the displacement $d(x) = h(x) - x$. Cancellation in the additive group gives

$$d(x+1) \neq d(x) \quad \Longleftrightarrow \quad h(x+1) \neq h(x) + 1.$$

Thus the set S of unsigned indices $0 \leq i < 2^n$ where $d(i+1) \neq d(i)$ has at most $2r$ members. The last index tests the wraparound edge and belongs to S exactly when its displacement changes. Lean proves the equivalence as `displacement_change_iff`, then injects these indices into the commutation-failure set in `displacementCuts_card`.

2.2 Reconstruction and its gate count

Let $\Delta_i = d(i+1) - d(i)$. For the unsigned representative of x, telescoping along the ordinary interval from 0 to x gives

$$d(x) = d(0) + \sum_{\substack{i \in S \\ i < x}} \Delta_i.$$

Lean first inserts zero summands at all indices outside S, restricts the sum to $0 \leq i < x$, and applies the finite telescoping identity. The wraparound correction never contributes, since its index $2^n - 1$ is not less than any unsigned word. This is `thresholdSum_displacement`.

The circuit proof follows that sum by induction over the finite set S. For each correction, compare the input with the hardwired threshold, select Δ_i or zero, and add the result to the previous partial

sum. The existing word-circuit lemmas charge respectively $5n+2$, $4n$, and $12n+1$ gates: $21n+3$ per correction. Two shared constant gates supply every bit of the hardwired words. One last addition recovers $h(x) = x + d(x)$, so the complete bound in `displacement_circuit` is

$$2 + |S|(21n + 3) + (12n + 1).$$

Substituting $|S| \leq 2r$ and relaxing the arithmetic gives $100(n + 1)(r + 1)$. Wiring and fanout are free; constants, comparisons, selectors and additions are all charged. The internal ripple-carry and multiplexer proofs are named prerequisites, provided by `Synthesis.word_ult`, `word_mux` and `word_add`.

2.3 Signed powers and charged changes of coordinates

For a permutation f, every integer power f^T is injective and commutes with f. Substituting $h = f^T$ proves `sparse_counter_power`; no dependence on the magnitude or sign of T enters the gate bound.

The final theorem allows counter coordinates to be supplied explicitly. Let f, g, e be permutations, with $e(g(x)) = e(x)+1$, and suppose circuits E and F compute e and e^{-1} on word bits. Conjugation sends g to c and preserves the number of disagreements: e bijects $D(f, g)$ with $D(efe^{-1}, ege^{-1})$. Apply the preceding theorem to efe^{-1}, then compose the resulting circuit between E and F. The homomorphism law for integer powers gives $(efe^{-1})^T = ef^Te^{-1}$, so the composite computes f^T. Circuit composition adds sizes, yielding

$$|E| + |F| + 100(n + 1)(|D(f, g)| + 1).$$

This is `sparse_coordinate_power`. It assumes both conversion circuits; it does not construct cheap coordinates for an arbitrary permutation. Sparse interval-chart construction, an exponent supplied as input, and the specialized navigation bounds remain subsequent developments.

References

[1] Sanjeev Arora and Boaz Barak. *Computational Complexity - A Modern Approach*. Cambridge University Press, 2009.

[2] Rutilo Moreno and Luis Manuel Rivera. Blocks in cycles and k-commuting permutations. *SpringerPlus*, 5:1949, 2016. doi: 10.1186/s40064-016-3638-7. URL https://doi.org/10.1186/s40064-016-3638-7. Article 1949.

[3] Christos H. Papadimitriou. *Computational Complexity*. Addison-Wesley, Reading, Massachusetts, 1994.

[4] Samuel Schlesinger. Square-root circuit bounds for iterated functions, 2026. URL https://github.com/SamuelSchlesinger/iterating-functions/tree/c4cfa198a241b3d2cc3bfa5f744a68a66f49be8b/lean. Source of ported Lean proofs, revision c4cfa198a241b3d2cc3bfa5f744a68a66f49be8b.